個人專案分享 · 2026 年 11 月

US$100、一週，做出一套地端程式碼問答系統

試了 6 種常見的 RAG 技巧，
最後只留下 **1** 個。

用 Claude Code 開發 · 模型與資料全部在本機執行 · 程式碼不離開電腦

讓 AI 從閉書考
變成開書考

一般 AI 問答 = 閉書考

- 只靠訓練時學到的知識
- 不知道你專案裡的程式碼
- 不知道時可能一本正經地亂編（幻覺）

RAG = 開書考

- 先從你的資料找出相關段落
- 只根據找到的段落回答
- 附上出處，可以自己查證

RAG（Retrieval-Augmented Generation，檢索增強生成）：先找資料，再回答。不用重新訓練模型，程式碼改了只要重建索引。

目標

用中文問程式碼，直接得到檔名與行號

「玩家受傷後的無敵時間在哪裡設定？」

條件：程式碼不能送到外部 AI 服務，模型與資料庫全部在本機執行

1 切片

程式碼依函式、class 切成小段

2 向量化

每段轉成一串數字，存進向量資料庫

3 檢索

問題也轉成數字，找出最相近的 4 段

4 回答

本機模型只根據這幾段回答，附出處

工具：Ollama（本機模型）· Qdrant（向量資料庫）· FastAPI · React · Docker

一週的成果

25
個 PR 合併

20
份決策紀錄

8
組對照實驗

我做什麼、AI 做什麼

Claude Code

- 寫程式、測試、文件
- 開 PR、修 CI
- 跑實驗、整理數據

我

- 決定方向與取捨
- 部署、除錯
- Review 每個 PR

最有用的一條規則

完成條件要可驗證：測試通過、CI 綠燈；數字只能來自實測，沒量過就留空，不准估算。

試了哪些技巧

六種常見的 RAG 技巧

分類取回

程式碼和文件分開挑，文件最多 1 段

混合檢索

「意思相近」加上「字面相同」一起找

相似度門檻

分數太低的片段直接丟掉

Rerank

多撈 10 段，再用另一個模型重新挑

防幻覺 prompt

要求片段沒提到就回答「找不到」

片段 metadata

加上檔名、class、常數名稱再向量化

起因：問「無敵時間在哪裡設定」，取回的 4 段全是說明文件，沒有程式碼。
做法：先做自動化評估，再每次只改一個開關比較。

實驗結果

只留下 1 個

分類取回 採用

文件佔比 95% → 25%，幾乎零成本

混合檢索 不採用

中文問題用不到，還把一個正確檔案擠掉

相似度門檻 0.70 不採用

套在程式碼上會丟掉 94% 的正確片段

Rerank 不採用

首字延遲多約 10 秒，改善不穩定

防幻覺 prompt 不採用

陷阱題都拒答了，但可回答的題目也誤拒 8/9

片段 metadata 不採用

沒有改善，延遲變長

學到的三件事

01

要有對照組

單看混合檢索，識別字題 33% → 100%，以為很有效；加了對照組才發現，是另一個開關的功勞。

02

通用建議要用自己的資料驗證

客服 RAG 常用的 0.70 門檻，套在程式碼上會丟掉 94% 的正確答案。網路上的數字，是別人的資料算出來的。

03

指標要成對看

防幻覺 prompt 的陷阱題拒答 5/5 看似成功；模型為了不亂編，乾脆什麼都說找不到，誤拒 8/9 才是真相。

給想嘗試的人

- **先寫清楚完成條件**：測試通過、CI 綠燈，AI 會自己修到達標
- **一個 PR 只做一件事**：好 review，也能平行開好幾個工作階段
- **把規則寫進 CLAUDE.md**：決策寫 ADR、問題寫 devlog，文件自然就有了
- **分清楚在哪裡跑**：雲端工作階段沒有 Docker，跑模型放 Codespaces 或本機
- **數字一定要有來源**：沒量過就留空

下一步

串接 AI 開發工具

改程式前先查相關程式碼，改得更準、減少幻覺；透過公司核准的 AI 工具

中文詞彙表

「連擊」→ combo、COMBO_STEP，讓中文問題也找得到程式碼